1.6 Simple Input and Output

In this section, we address the basics of input and output in Python, describing standard input and output through the user console, and Python's support for reading and writing text files.

1.6.1 Console Input and Output

The print Function

The built-in function, print, is used to generate standard output to the console. In its simplest form, it prints an arbitrary sequence of arguments, separated by spaces, and followed by a trailing newline character. For example, the command print('maroon', 5) outputs the string 'maroon 5\n'. Note that arguments need not be string instances. A nonstring argument x will be displayed as str(x). Without any arguments, the command print( ) outputs a single newline character.

The print function can be customized through the use of the following keyword parameters (see Section 1.5 for a discussion of keyword parameters):

- By default, the print function inserts a separating space into the output between each pair of arguments. The separator can be customized by providing a desired separating string as a keyword parameter, sep. For example, colon-separated output can be produced as print(a, b, c, sep=':'). The separating string need not be a single character; it can be a longer string, and it can be the empty string, sep='', causing successive arguments to be directly concatenated.
- By default, a trailing newline is output after the final argument. An alternative trailing string can be designated using a keyword parameter, end. Designating the empty string end='' suppresses all trailing characters.
- By default, the print function sends its output to the standard console. However, output can be directed to a file by indicating an output file stream (see Section 1.6.2) using file as a keyword parameter.

The input Function

The primary means for acquiring information from the user console is a built-in function named input. This function displays a prompt, if given as an optional parameter, and then waits until the user enters some sequence of characters followed by the return key. The formal return value of the function is the string of characters that were entered strictly before the return key (i.e., no newline character exists in the returned string).

When reading a numeric value from the user, a programmer must use the input function to get the string of characters, and then use the int or float syntax to construct the numeric value that character string represents. That is, if a call to response = input( ) reports that the user entered the characters, '2013', the syntax int(response) could be used to produce the integer value 2013. It is quite common to combine these operations with a syntax such as

```
year = int(input('In what year were you born? '))
```

if we assume that the user will enter an appropriate response. (In Section 1.7 we discuss error handling in such a situation.)

Because input returns a string as its result, use of that function can be combined with the existing functionality of the string class, as described in Appendix A. For example, if the user enters multiple pieces of information on the same line, it is common to call the split method on the result, as in

```
reply = input('Enter x and y, separated by spaces: ')
pieces = reply.split( )    # returns a list of strings, as separated by spaces
x = float(pieces[0])
y = float(pieces[1])
```

A Sample Program

Here is a simple, but complete, program that demonstrates the use of the input and print functions. The tools for formatting the final output is discussed in Appendix A.

```
age = int(input('Enter your age in years: '))
max_heart_rate = 206.9 - (0.67 * age)    # as per Med Sci Sports Exerc.
target = 0.65 * max_heart_rate
print('Your target fat-burning heart rate is', target)
```

1.6.2 Files

Files are typically accessed in Python beginning with a call to a built-in function, named open, that returns a proxy for interactions with the underlying file. For example, the command, fp = open('sample.txt'), attempts to open a file named sample.txt, returning a proxy that allows read-only access to the text file.

The open function accepts an optional second parameter that determines the access mode. The default mode is 'r' for reading. Other common modes are 'w' for writing to the file (causing any existing file with that name to be overwritten), or 'a' for appending to the end of an existing file. Although we focus on use of text files, it is possible to work with binary files, using access modes such as 'rb' or 'wb'.

When processing a file, the proxy maintains a current position within the file as an offset from the beginning, measured in number of bytes. When opening a file with mode `'r'` or `'w'`, the position is initially 0; if opened in append mode, `'a'`, the position is initially at the end of the file. The syntax fp.close( ) closes the file associated with proxy fp, ensuring that any written contents are saved. A summary of methods for reading and writing a file is given in Table 1.5

Calling Syntax	Description
fp.read( )	Return the (remaining) contents of a readable file as a string.
fp.read(k)	Return the next k bytes of a readable file as a string.
fp.readline( )	Return (remainder of) the current line of a readable file as a string.
fp.readlines( )	Return all (remaining) lines of a readable file as a list of strings.
for line **in** fp:	Iterate all (remaining) lines of a readable file.
fp.seek(k)	Change the current position to be at the kth byte of the file.
fp.tell( )	Return the current position, measured as byte-offset from the start.
fp.write(string)	Write given string at current position of the writable file.
fp.writelines(seq)	Write each of the strings of the given sequence at the current position of the writable file. This command does *not* insert any newlines, beyond those that are embedded in the strings.
print(..., file=fp)	Redirect output of print function to the file.

Table 1.5: Behaviors for interacting with a text file via a file proxy (named fp).

Reading from a File

The most basic command for reading via a proxy is the read method. When invoked on file proxy fp, as fp.read(k), the command returns a string representing the next k bytes of the file, starting at the current position. Without a parameter, the syntax fp.read( ) returns the remaining contents of the file in entirety. For convenience, files can be read a line at a time, using the readline method to read one line, or the readlines method to return a list of all remaining lines. Files also support the for-loop syntax, with iteration being line by line (e.g., **for** line **in** fp:).

Writing to a File

When a file proxy is writable, for example, if created with access mode `'w'` or `'a'`, text can be written using methods write or writelines. For example, if we define fp = open(`'results.txt'`, `'w'`), the syntax fp.write(`'Hello World.\n'`) writes a single line to the file with the given string. Note well that write does not explicitly add a trailing newline, so desired newline characters must be embedded directly in the string parameter. Recall that the output of the print method can be redirected to a file using a keyword parameter, as described in Section 1.6.